

Android Architecture Overview

Introduction

- Android is an open-source, Linux-based operating system designed for mobile devices.
- Developed for Battery Powered low memory devices (Smartphones and tabs)
- Originally developed by Android Inc., acquired by Google in 2005.
- Beyond smartphones and tablets, Android powers devices like Android Auto, TVs, watches, and cameras.
- Its architecture is modular, allowing for flexibility and scalability.
- Understanding its components is crucial for efficient app development.

Why Android is Popular

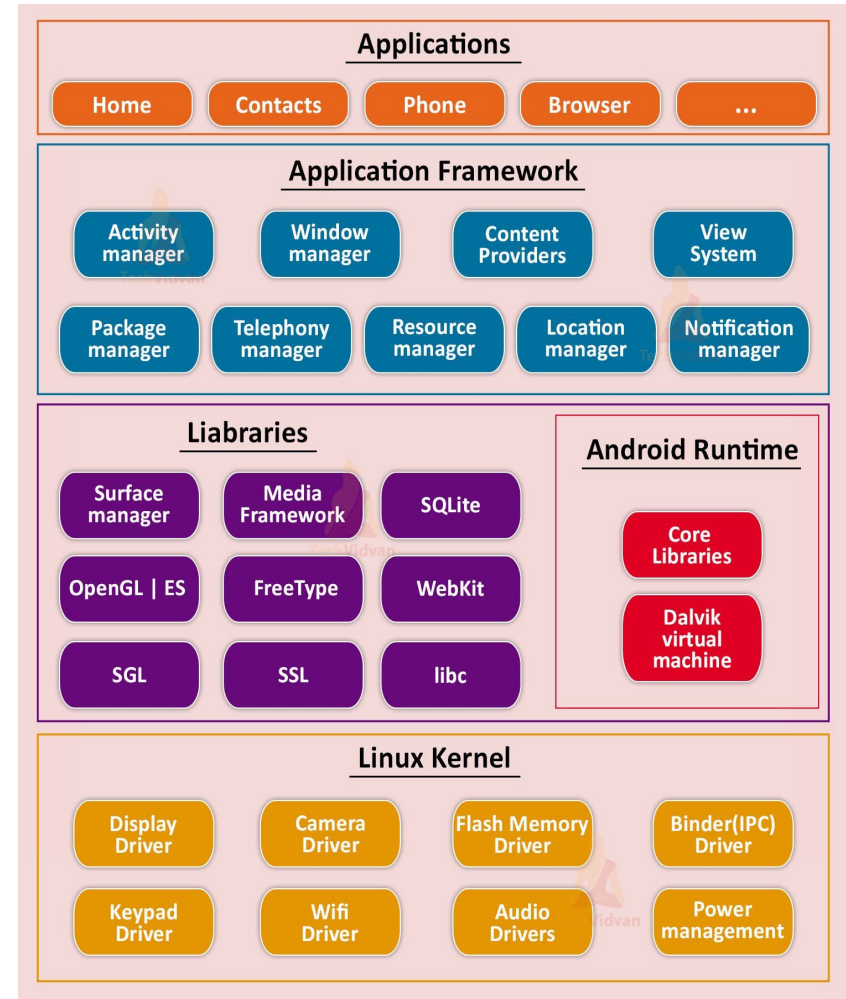


World Before Android

- Before Android 2 major leaders in smartphone industry
 - Apple
 - BlackBerry
- Apple introduces their 1st touch screen smartphone in 2007
 - Apple iPhone 7 (Running iOS)
 - Best Performance (Own Hardware/ Own OS)
- Other Companies striving hard for OS
 - Support for touch screen
 - Optimized according to hardware for better performance
 - Difficult task for other mobile companies

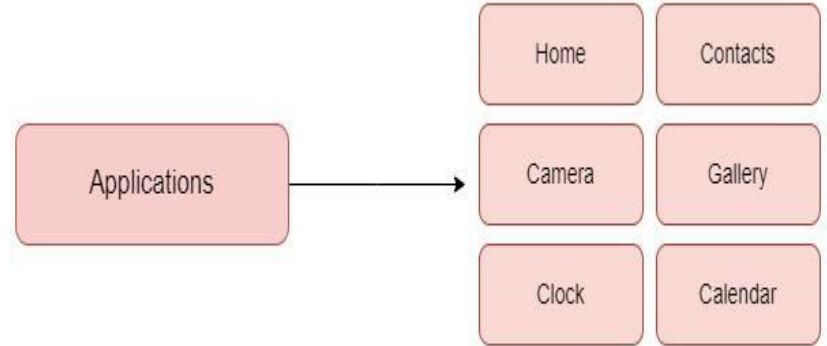
Android Architecture Diagram

- The main components of Android architecture are the following
 - Applications
 - Application Framework
 - Android Runtime
 - Platform Libraries
 - Linux Kernel



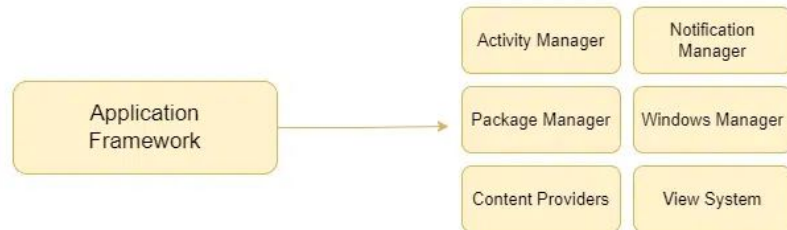
Applications

- This is the topmost layer where user-facing applications reside.
- Includes both system apps (e.g., Contacts, Camera) and third-party apps from the Play Store.
- Runs within the Android Runtime using services provided by the Application Framework.



Application Framework

- Provides high-level services to applications.
- Key Components:
 - Activity Manager: Manages app activities and their lifecycle.
 - Notification Manager: Handles notifications.
 - Package Manager: Manages installed applications.
 - Window Manager: Controls window placement and appearance.
 - Content Providers: Facilitates data sharing between apps.
 - View System: Manages UI elements like buttons and text fields.

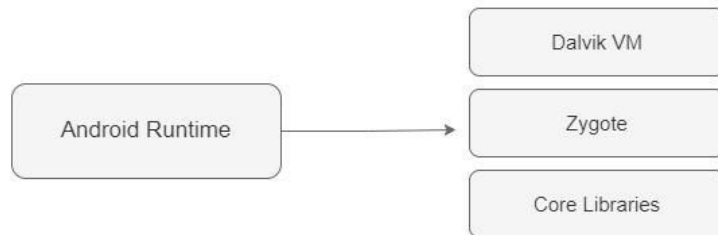


Android Runtime

- Provides the environment for running applications.

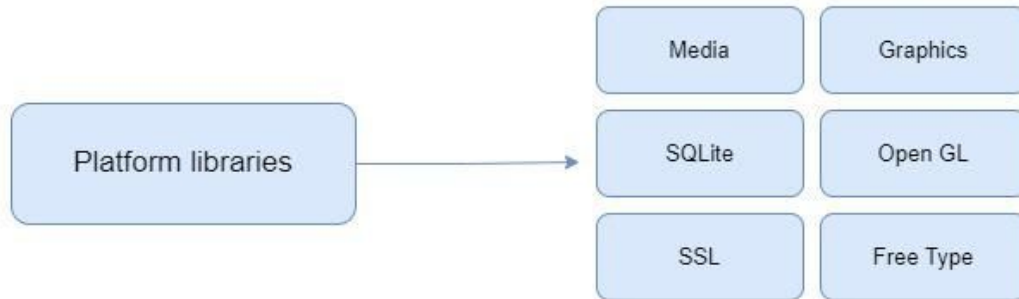
Components:

- **Core Libraries:** Offer essential Java and Kotlin libraries.
- **Dalvik Virtual Machine (DVM):** Executes applications in older Android versions.
- **Android Runtime (ART):** Replaced DVM in Android 5.0 and above; uses ahead-of-time (AOT) compilation for improved performance.
- The **Zygote** is a fundamental process in the Android operating system, serving as the parent for all application processes and the System Server. Its primary function is to enable fast and efficient app launching by preloading essential resources and classes.

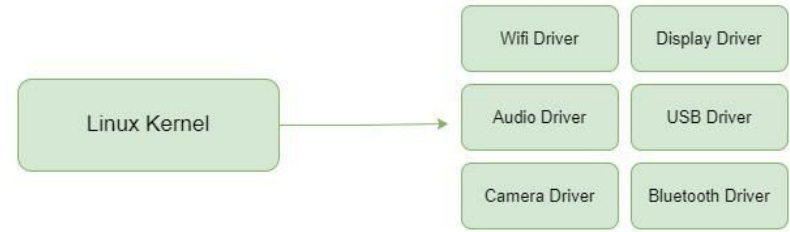


Platform Libraries

- Consists of C/C++ libraries providing essential functionalities.
- Examples:
 - WebKit: Browser engine.
 - SQLite: Database management.
 - SSL Libraries: Security protocols.
 - Media Libraries: Audio and video playback



Linux Kernel



- The foundational layer of Android.
- Manages hardware interactions, memory, and process control.
- Provides drivers for hardware components like camera, display, and sensors.
- The features of Linux kernel are:
 - **Security:** The Linux kernel handles the security between the application and the system.
 - **Memory Management:** It efficiently handles the memory management thereby providing the freedom to develop our apps.
 - **Process Management:** It manages the process well, allocates resources to processes whenever they need them.
 - **Network Stack:** It effectively handles the network communication.
 - **Driver Model:** It ensures that the application works properly on the device and hardware manufacturers responsible for building their drivers into the Linux build.

Android Programming Languages

- Primary Languages:
 - Java: Traditional language for Android development.
 - Kotlin: Modern, preferred language for Android development.
- Markup Language:
 - XML: Used for designing user interfaces and layouts.

Core Android Components

- **Activity:**
 - Visible screen which runs on foreground
 - **Views:** controls for interaction
 - **Intents:** Share data from on app to other app
- **Services:**
 - Perform background operations without user interaction.
 - eg. whatsapp
- **Broadcast Receivers:**
 - Respond to system-wide broadcast announcements.
 - eg. low battery, incoming call while listening songs
- **Content Providers:**
 - Manage shared data between applications.
 - Eg. From whatsapp we share contacts which is available

Activities
Views/User Interface
Intents
Services
Content Provider
Broadcast Receiver

Activities

- It deals with the UI and the user interactions to the screen.
- For example, an email application might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails.
- If an application has more than one activity, then one of them should be marked as the activity that is presented when the application is launched.
- The activity is implemented through the following.

```
class MainActivity : AppCompatActivity() {  
  
}
```

Services

- A service is a component that runs in the background to perform long-running operations.
- For example, a service might play music in the background while the user is in a different application, or it might fetch data over the network without blocking user interaction with an activity.
- The main purpose of the Services is to provide non-stop working of the app without breaking any interaction with the user.

```
class ServiceName : Service() {  
  
}
```

Broadcast Receivers

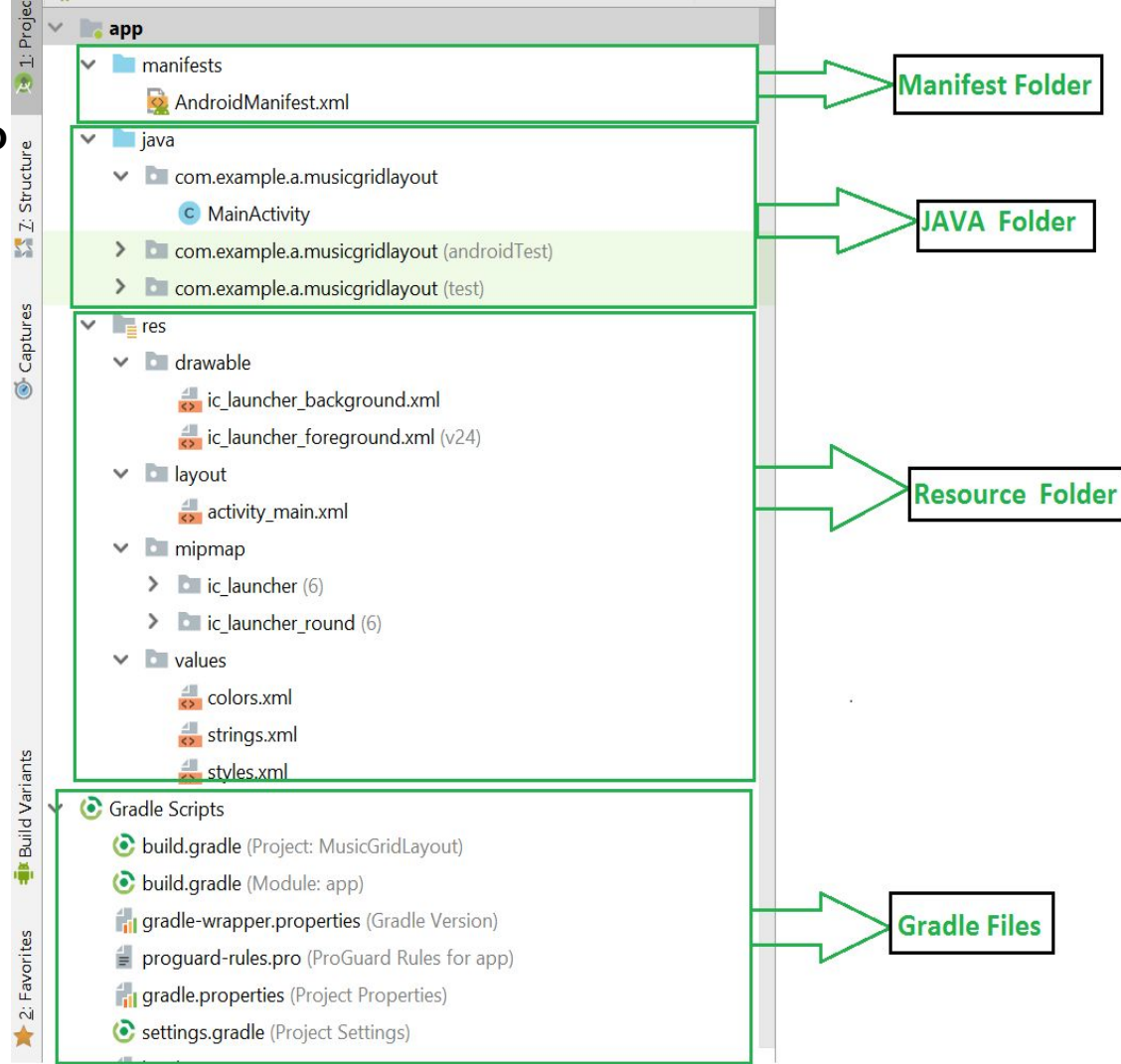
- A Broadcast is used to respond to messages from other applications or from the System.
- For example, when the battery of the phone is low, then the Android OS fires a Broadcasting message to launch the Battery Saver function or app, after receiving the message the appropriate action is taken by the app.
- Broadcast Receiver is the subclass of BroadcastReceiver class and each object is represented by Intent objects.

Content Providers

- A content provider component supplies data from one application to others on request.
- Such requests are handled by the methods of the ContentResolver class.
- The data may be stored in the file system, the database or somewhere else entirely.
- A content provider is implemented as a subclass of ContentProvider class and must implement a standard set of APIs that enable other applications to perform transactions.

```
class contentProviderName : ContentProvider() {  
    override fun onCreate(): Boolean {}  
}
```

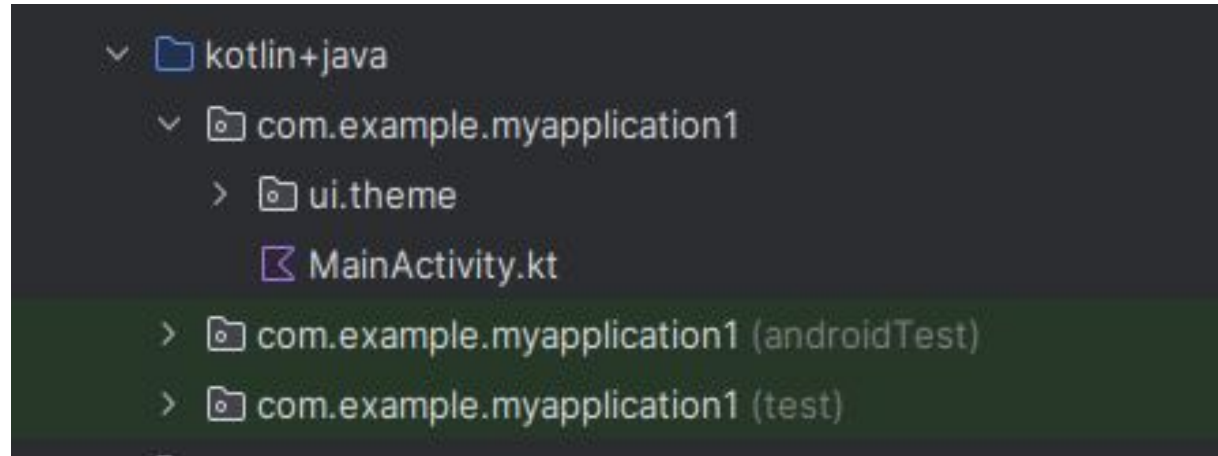
Structural Layout Of Android Studio



Project Directory Structure

- Three Parts
 - Manifest File
 - Java+Kotlin: Business Logic
 - Resources: Additional Resources

Java+Kotlin Folder



1 Usage

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            MyApplication1Theme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(paddingValues = innerPadding)  
                    )  
                }  
            }  
        }  
        val toast = Toast.makeText(applicationContext, text = "onCreate Called", duration = Toast.LENGTH_LONG).show()  
    }  
}
```

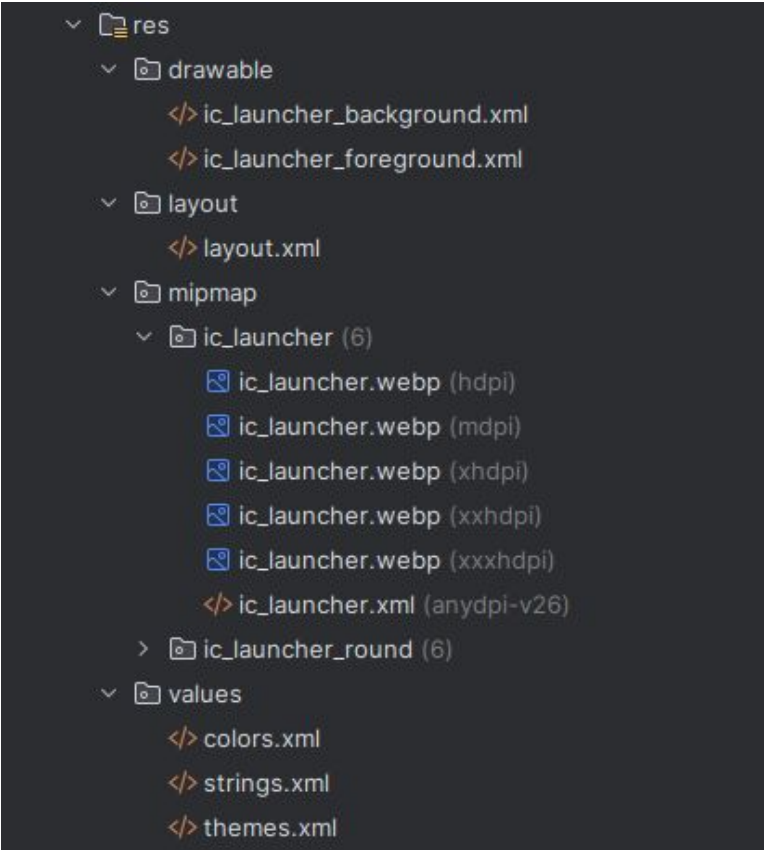
2 Usages

```
@Composable  
fun Greeting(name: String, modifier: Modifier = Modifier) {  
    Text(  
        text = "Hello $name!",  
        modifier = modifier  
    )  
}
```

```
@Preview(showBackground = true)  
@Composable  
fun GreetingPreview() {  
    MyApplication1Theme {  
        Greeting(name = "Android")  
    }  
}
```

Resources Folder

- Drawable: Contains images, logos, icons
- Layout:
 - Xml layout files used to define the UI app
 - Atleast one layout xml file per activity
- Mipmap:
 - Contains app/launcher icons shown on home screen
 - Different density type of icons
- Values:
 - Contains various xml files containing values such as
 - Strings colors, style etc
 - Declare once use anywhere



```

▼ res
  ▼ drawable
    </> ic_launcher_background.xml
    </> ic_launcher_foreground.xml
  ▼ layout
    </> layout.xml
  ▼ mipmap
    ▼ ic_launcher (6)
      ic_launcher.webp (hdpi)
      ic_launcher.webp (mdpi)
      ic_launcher.webp (xhdpi)
      ic_launcher.webp (xxhdpi)
      ic_launcher.webp (xxxhdpi)
      </> ic_launcher.xml (anydpi-v26)
    > ic_launcher_round (6)
  ▼ values
    </> colors.xml
    </> strings.xml
    </> themes.xml

```

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

```
<resources>
     <string name="app_name">My Application</string>
</resources>
```

Manifest Folder

- Contains a manifest file **AndroidManifest.xml**
- Most important file for any android application
- Contains information about application such as
 - Defines structure of application
 - Metadata about application like application name, version, sdk version
 - Requirements of applications like access permissions
 - Components of application
- Mediator between our application and android OS

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="My Application"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="My Application"
            android:theme="@style/Theme.MyApplication">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Root element

Defines behaviour
of activity

Every activity starts with .

1st activity visible when
app is launched

Launch on home
screen

Adding App Components in Manifest

- **<activity>** : for each subclass of Activity
- **<service>** : for each subclass of Service
- **<receiver>** : for each subclass of BroadcastReceiver
- **<provider>** : for each subclass of ContentProvider

Permissions

- Android app must request permissions to access sensitive user data (contacts/sms/gallery)
- Android app must request permissions to access features like camera, internet access
- Developer must write these request permissions in Manifest.xml file
- User can approve or reject some app permissions at runtime

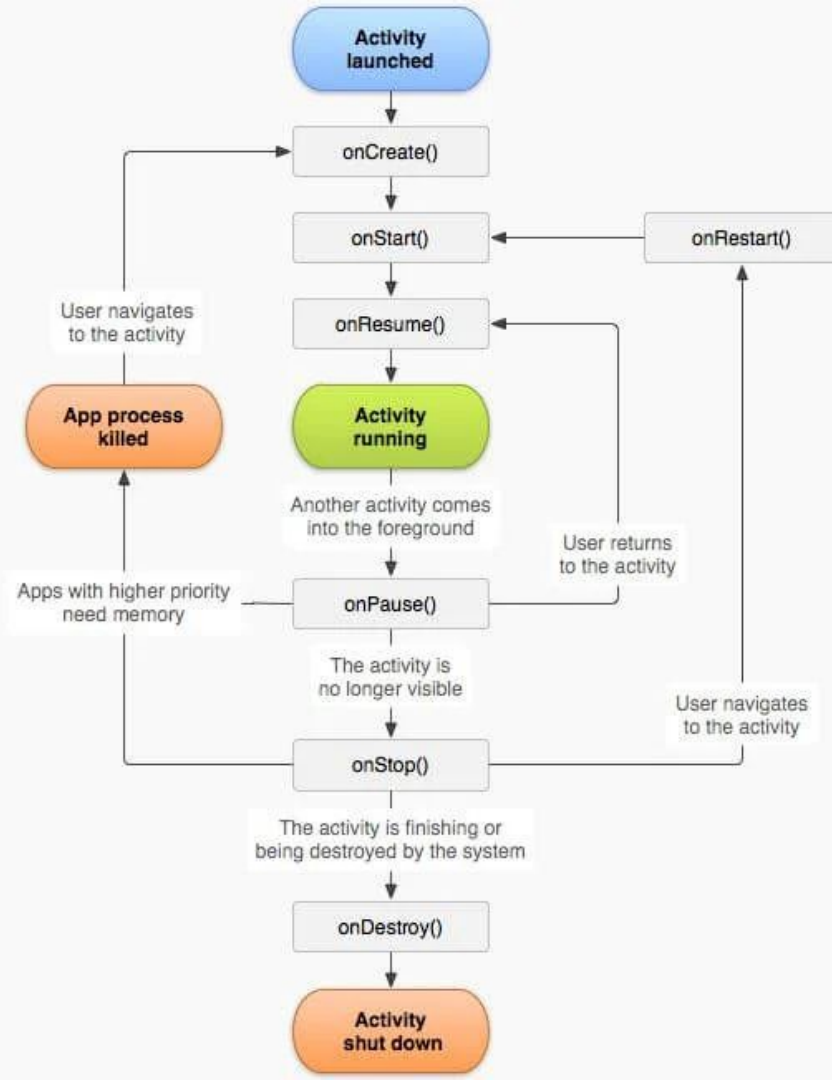
```
<uses-permission android:name="android.permission.INTERNET" />
```

```
<uses-permission android:name="android.permission.CAMERA" />
```

Gradle Scripts

- Automated build system
- Defines build configurations and dependencies for app modules or specific to one app module .

Activity Life Cycle



```
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Bundle containing previous frozen state
        setContentView(R.layout.activity_main)

        // The content view pointing to the id of layout
        // in the file activity_main.xml
        val toast = Toast.makeText(applicationContext, "onCreate Called", Toast.LENGTH_LONG).show()
    }
}
```

```
override fun onStart() {  
    super.onStart()  
    // It will show a message on the screen  
    // then onStart is invoked  
    val toast = Toast.makeText(applicationContext, "onStart Called", Toast.LENGTH_LONG).show()  
}
```

```
override fun onRestart() {  
    super.onRestart()  
    // It will show a message on the screen  
    // then onRestart is invoked  
    val toast = Toast.makeText(applicationContext, "onRestart Called",  
Toast.LENGTH_LONG).show()  
}
```

```
override fun onResume() {  
    super.onResume()  
    // It will show a message on the screen  
    // then onResume is invoked  
    val toast = Toast.makeText(applicationContext, "onResume Called", Toast.LENGTH_LONG).show()  
}  
}
```

```
override fun onPause() {  
    super.onPause()  
    // It will show a message on the screen  
    // then onPause is invoked  
    val toast = Toast.makeText(applicationContext, "onPause Called", Toast.LENGTH_LONG).show()  
}  
}
```

```
override fun onStop() {  
    super.onStop()  
    // It will show a message on the screen  
    // then onStop is invoked  
    val toast = Toast.makeText(applicationContext, "onStop Called", Toast.LENGTH_LONG).show()  
}  
}
```

```
protected void onDestroy() {  
    // It will show a message on the screen  
    // then onDestroy is invoked  
    Toast toast = Toast.makeText(getApplicationContext(), "onDestroy Called",  
    Toast.LENGTH_LONG).show();  
}  
}
```

<https://media.geeksforgeeks.org/wp-content/uploads/20191125171002/Activity-Lifecycle-in-Android-Demo-App.mp4>

Logging in Android (Logcat)

- Window and command line tool
- Used to see system and app generated logs
- Every app/system generates logs about events/ errors occurring in the system
- Logs are useful for debugging purpose
- Logcat allows to see the logs messages generated by the system or application

Generating own logs

- Android sdk provides a built in class for generating log messages
 - **android.util.Log extends java.lang.Object**
- Provides 6 methods for generating logs
 - **Log.d(tag: String, msg: String) (Debug)**:debugging information, helpful during development to track the flow of execution
 - **Log.e(tag: String, msg: String) (Error)**: error conditions, indicating serious problems that might lead to application crashes or incorrect behavior.
 - **Log.i(tag: String, msg: String) (Information)**: informational messages, indicating significant events or states in the application
 - **Log.v(tag: String, msg: String) (Verbose)**: detailed information that may be useful during development
 - **Log.w(tag: String, msg: String) (Warning)**: log potential issues or unexpected but non-critical conditions.
 - **Log.wtf(tag: String, msg: String) (What a Terrible Failure)**: very serious errors, indicating a condition that should never happen. (Exceptions)

Android Studio interface showing the MainActivity.kt file and the Logcat output.

MainActivity.kt

```
17 class MainActivity : AppCompatActivity() {
33 }
34
35 @Composable
36 fun Greeting(name: String, modifier: Modifier = Modifier) {
37     Text(
38         text = "Hello $name!",
39         modifier = modifier
40     )
41     Log.e("Mytag", "This is simple debug msg")
42 }
43
44 @Preview(showBackground = true)
45 @Composable
46 fun GreetingPreview() {
47     MyApplicationTheme {
48         Greeting("Android")
49     }
50 }
```

Logcat

Logcat x +

Medium Phone API 36.0 (emulator-5554) Android 16, v mytag

----- PROCESS STARTED (8488) for package com.example.myapplication -----
2025-09-11 11:08:15.164 8488-8488 Mytag com.example.myapplication D This is simple debug msg
----- PROCESS ENDED (8488) for package com.example.myapplication -----
----- PROCESS STARTED (8858) for package com.example.myapplication -----
2025-09-11 11:09:58.780 8858-8858 Mytag com.example.myapplication E This is simple debug msg

Install successfully finished in 269 ms